

CORSO DI CYBERSECURITY – EPICODE

ESERCITAZIONE D'ESAME

Esercizio 2: Studio IoC

Analisi comportamentale di un campione malevolo
distribuito tramite GitHub

A cura di:

Nicola Madonna

Epicode

4 settembre 2026

Indice

1	Riassunto	2
2	Obiettivi	2
3	Metodologia	3
4	Analisi dei Dati Raccolti	4
4.1	Panoramica del verdetto e statistiche del task	4
4.2	Catena di infezione: dal browser al payload	5
4.3	Albero dei processi	5
4.4	Esecuzione indiretta tramite InstallUtil.exe	6
4.5	Comando e controllo	7
4.6	Firme di rete Suricata	9
4.7	Connessione anomala verso l'infrastruttura di GitHub	9
4.8	File droppati e identificazione degli hash reali	11
4.9	Verifica su VirusTotal e discrepanza degli hash	12
4.10	Terminazione anomala e interpretazione dell'exit code	13
4.11	Indicatori di compromissione	14
4.12	Ipotesi sulla famiglia di appartenenza	15
5	Conclusioni	16

1 Riassunto

L'esercizio ha richiesto lo studio di un task pubblico della sandbox online ANY.RUN (9a158718-43fe-45ce-85b3-66203dbc2281) e la redazione di un report che spiegasse le minacce osservate. Il task, eseguito il 25 agosto 2024 su una macchina virtuale Windows 10 Professional a 64 bit, riproduce lo scenario di un utente che naviga su GitHub e scarica due eseguibili ospitati nei repository `kioluu` e `frew` dell'account MELITERRER: `Jvczfhe.exe` e `Muadnrd.exe`.

L'analisi ha permesso di ricostruire l'intera catena di infezione: il browser scarica ed esegue i due binari, entrambi protetti con l'offuscatore commerciale .NET Reactor e con i metadati PE contraffatti per apparire come componenti di Microsoft Edge. All'avvio ciascun campione lancia un ritardo artificiale tramite `cmd /c timeout 21` per eludere le sandbox automatiche, quindi utilizza il binario legittimo di sistema `InstallUtil.exe` come vettore di esecuzione indiretta. Il processo così creato stabilisce un canale di comando e controllo verso il dominio Dynamic DNS `egehgdhjbhjtire.duckdns.org`, risolto sull'indirizzo `91.92.253.47` (Bulgaria), sulla porta non standard `7702`. Entrambi i campioni terminano con un crash gestito da `WerFault.exe` e con exit code `3762504530 (0xE0434352)`, valore che identifica un'eccezione non gestita del CLR .NET.

Il report documenta inoltre una discrepanza rilevante emersa in fase di verifica: l'hash riportato da ANY.RUN come oggetto principale del task non corrisponde al payload effettivamente droppato, ma a un file di testo di 56 byte residuo di un download interrotto. Tale hash risulta pulito su VirusTotal (0/62), mentre gli hash reali dei due eseguibili, ricavati dalla sezione *Dropped files*, sono quelli da utilizzare come indicatori di compromissione.

2 Obiettivi

- Studiare il task ANY.RUN indicato nella traccia e comprendere il comportamento del campione analizzato.
- Ricostruire la catena di infezione completa, dalla fase di consegna (delivery) fino al contatto con l'infrastruttura di comando e controllo.
- Identificare le tecniche di evasione, di offuscamento e di esecuzione indiretta adottate dal malware.
- Estrarre e validare gli indicatori di compromissione (IoC) di tipo host e di tipo rete, verificandone la coerenza tramite fonti terze (VirusTotal).
- Documentare eventuali limitazioni dell'analisi e le discrepanze riscontrate nei dati forniti dalla piattaforma.

3 Metodologia

L'analisi è stata condotta interamente sulla piattaforma ANY.RUN, una sandbox interattiva che esegue il campione in una macchina virtuale isolata (Windows 10 Professional, 64 bit) registrando in tempo reale processi, registro di sistema, attività sul file system e traffico di rete. Non è stata quindi allestita alcuna VM locale: l'esercizio consiste nello studio di un task già eseguito e reso pubblico (9a158718-43fe-45ce-85b3-66203dbc2281, 25 agosto 2024, durata 315 secondi), consultato in modalità *Guest*.

Il task non è stato avviato sottomettendo un file, ma l'URL `github.com/MELITERRER/frew/blob/main/Jvczfhe.exe`: la sandbox ha aperto Firefox su tale indirizzo, riproducendo la navigazione di una vittima reale. Nel corso della sessione sono stati scaricati ed eseguiti due binari distinti, `Jvczfhe.exe` e `Muadnrd.exe`, provenienti da due repository differenti dello stesso account GitHub (`frew` e `kioluu`).

Le opzioni di rete del task sono determinanti per interpretare correttamente i dati raccolti, perché stabiliscono se il traffico osservato sia reale o simulato.

Tabella 1: Opzioni di configurazione del task e loro impatto sull'analisi.

Opzione	Stato	Implicazione
Network	on	Il traffico verso il C2 è reale, non simulato
FakeNet	off	Nessuna risposta artificiale ai domini contattati
MITM proxy	off	Il contenuto del traffico TLS non è ispezionabile
Route via Tor	off	L'IP sorgente è quello reale della sandbox
Autoconferma UAC	on	Le richieste di elevazione sono accettate
Privacy	public	Il task è consultabile pubblicamente

La combinazione *Network on* con *FakeNet off* è particolarmente significativa: la risoluzione DNS di `egehgdhjbjhtre.duckdns.org` e la connessione TCP sulla porta 7702 sono avvenute realmente verso l'infrastruttura dell'attaccante. Al contrario, l'assenza di un proxy MITM impedisce di osservare il contenuto in chiaro delle comunicazioni cifrate, limitando l'analisi al livello dei metadati di connessione.

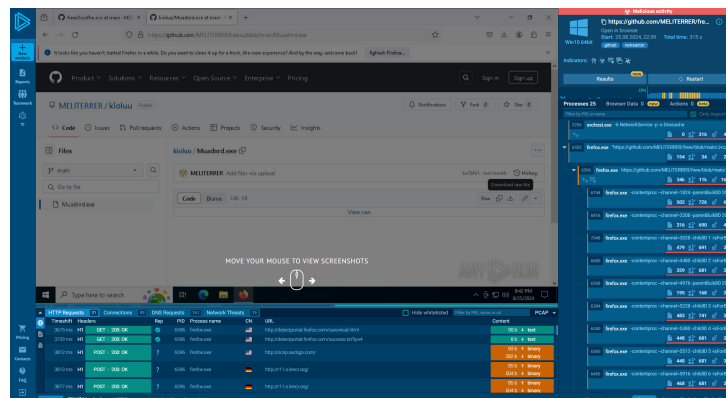


Figura 1: Interfaccia ANY.RUN: pagina GitHub di origine, elenco processi con verdetto *Malicious activity* e richieste HTTP registrate.

4 Analisi dei Dati Raccolti

Una volta completata la sessione, la sandbox restituisce un quadro comportamentale articolato. Il verdetto complessivo è *Malicious activity*, assegnato sulla base di sette indicatori sospetti e di diciannove eventi di rete classificati dal motore di firme Suricata. Nelle sottosezioni che seguono i dati vengono esaminati per dominio di analisi (processi, rete, registro, file system) e successivamente ricomposti in una cronologia unitaria e in una tabella di indicatori.

4.1 Panoramica del verdetto e statistiche del task

Tabella 2: Statistiche complessive del task ANY.RUN.

Metrica	Valore
Processi totali	155
Processi monitorati	25
Processi classificati come <i>malicious</i>	0
Processi classificati come <i>suspicious</i>	3
Eventi di registro totali	35 308
di cui in lettura	35 167
di cui in scrittura	140
di cui in cancellazione	1
File eseguibili creati	6
File sospetti creati	190
Richieste HTTP(S)	31
Connessioni TCP/UDP	99
Richieste DNS	161
Minacce di rete rilevate	19

Il dato più interessante di questa tabella è apparentemente contraddittorio: zero processi classificati come *malicious* a fronte di un verdetto complessivo di attività malevola. Il dato grezzo del report chiarisce in parte l'apparente contraddizione: la sezione *Malicious* elenca zero voci, mentre la sezione *Suspicious* ne elenca sette, e la sezione *Malware configuration* risulta vuota (“No data”). Il verdetto complessivo sembra quindi derivare dall'aggregazione degli indicatori comportamentali sospetti piuttosto che dal riconoscimento certo di una famiglia nota – coerentemente con il fatto che l'offuscamento tramite .NET Reactor ha impedito l'identificazione statica del campione. Il meccanismo esatto con cui ANY.RUN pesa e combina questi indicatori nel verdetto finale non è documentato nei dati del task e non viene quindi ricostruito qui; ciò che resta comunque un promemoria utile per l'analista è che l'assenza di una firma non è prova di innocuità.

4.2 Catena di infezione: dal browser al payload

La catena di consegna sfrutta GitHub come piattaforma di hosting. Questa scelta non è casuale e rappresenta il primo elemento di interesse dell'analisi: il dominio `github.com` gode di ottima reputazione, è raramente inserito in blacklist aziendali e serve i contenuti tramite HTTPS con certificati validi. Un download da GitHub supera quindi la maggior parte dei controlli perimetrali basati su reputazione del dominio.

La sequenza ricostruita è la seguente:

1. `explorer.exe` avvia `firefox.exe` (PID 6552) sull'URL del repository; il processo genera a sua volta l'istanza principale del browser (PID 6596) e i consueti processi figli di tipo *contentproc*.
2. Il browser scarica i file nella cartella `C:\Users \admin\Downloads`, creando prima i file temporanei con estensione `.part` e poi rinominandoli.
3. A ciascun file scaricato viene associato un flusso alternativo `Zone.Identifier`, il cosiddetto *Mark of the Web*, che marca il file come proveniente da Internet.
4. L'utente esegue manualmente i due binari, che ANY.RUN registra come processi figli di `firefox.exe`.

La presenza del *Mark of the Web* è un dettaglio rilevante in ottica difensiva: Windows avrebbe normalmente mostrato l'avviso SmartScreen prima dell'esecuzione di un file scaricato e non riconosciuto, sebbene i dati del task non permettano di confermare se tale avviso sia effettivamente comparso. La conferma più solida che la catena richieda un'interazione esplicita della vittima viene comunque dal dato di processo già osservato: i due binari non vengono eseguiti all'interno del processo del browser (come avverrebbe con un exploit drive-by che sfrutta una vulnerabilità del renderer), ma come processi distinti lanciati dalla cartella Downloads, il che presuppone un'azione volontaria dell'utente. Il vettore iniziale è quindi di tipo *social engineering* e non un exploit automatico.

4.3 Albero dei processi

L'albero dei processi è il cuore dell'analisi comportamentale. I processi rilevanti sono riepilogati nella tabella seguente, con relazione padre-figlio.

Tabella 3: Processi rilevanti del task, con relazione padre-figlio.

PID	Processo	Riga di comando	Padre
6596	firefox.exe	URL del repository GitHub	firefox.exe (6552)
7492	Jvczfhe.exe	"...\Downloads\Jvczfhe.exe"	firefox.exe
7520	cmd.exe	/c timeout 21 & exit	Jvczfhe.exe
7572	timeout.exe	timeout 21	cmd.exe
5152	InstallUtil.exe	"...\v4.0.30319\InstallUtil.exe"	Jvczfhe.exe
1356	WerFault.exe	-u -p 7492 -s 2676	Jvczfhe.exe
7824	Muadnrd.exe	"...\Downloads\Muadnrd.exe"	firefox.exe
7876	cmd.exe	/c timeout 21 & exit	Muadnrd.exe
7968	timeout.exe	timeout 21	cmd.exe
7248	Muadnrd.exe	"...\Downloads\Muadnrd.exe"	Muadnrd.exe
7584	WerFault.exe	-u -p 7824 -s 2888	Muadnrd.exe
2256	svchost.exe	-k NetworkService -p -s Dnscache	services.exe

Il confronto tra i due rami evidenzia un comportamento sostanzialmente identico: stesso schema di avvio, stessa riga di comando per il ritardo, stessa terminazione anomala. I due file sono quindi due varianti dello stesso loader, distribuite con nomi casuali su repository differenti dello stesso account — una tecnica di ridondanza che aumenta la resilienza della campagna alla rimozione di un singolo repository.

Una differenza esiste ed è significativa: `Muadnrd.exe` genera una seconda istanza di sé stesso (PID 7248), comportamento che ANY.RUN classifica come *Application launched itself*. Questo pattern è tipico dei loader che, dopo aver decifrato in memoria il payload effettivo, riavviano il proprio eseguibile per eseguirlo in un contesto “pulito”. Non a caso è proprio sulla seconda istanza (PID 7248), e non sulla prima, che la piattaforma rileva la firma di .NET Reactor.

4.4 Esecuzione indiretta tramite InstallUtil.exe

L'elemento tecnicamente più rilevante dell'intera catena è l'esecuzione di:

```
C:\Windows\Microsoft.NET\Framework\v4.0.30319\InstallUtil.exe
```

`InstallUtil.exe` è un'utilità legittima di Microsoft, firmata digitalmente e distribuita con il .NET Framework, il cui scopo ordinario è installare componenti di servizio da un assembly .NET. Rientra nella categoria dei *LOLBin* (*Living Off the Land Binaries*): programmi già presenti nel sistema e considerati affidabili, che gli attaccanti riutilizzano per eseguire codice arbitrario senza depositare strumenti propri sul disco.

Il vantaggio per l'attaccante è duplice. In primo luogo, l'esecuzione avviene sotto l'identità di un processo Microsoft firmato, il che elude i controlli basati su *application whitelisting* e sulla validazione del certificato digitale. In secondo luogo, se il codice malevolo viene eseguito senza mai essere scritto su disco in forma eseguibile autonoma, l'analisi statica basata su file risulta inefficace. I dati del task non includono un'ispezione diretta della memoria di processo (ad esempio un dump analizzato o un log delle chiamate di iniezione), per cui quest'ultimo punto resta un'inferenza, seppure la più coerente con l'insieme delle prove disponibili.

Due elementi confermano che si tratta di un abuso e non di un uso legittimo. Il primo è la relazione di parentela: `InstallUtil.exe` viene avviato da `Jvczfhe.exe`, un eseguibile scaricato da Internet e residente nella cartella Downloads, mentre un uso legittimo lo vedrebbe invocato da un installer o da un prompt amministrativo. Il secondo è l'assenza totale di argomenti: invocato senza il percorso di un assembly da installare, `InstallUtil.exe` non ha alcuna funzione legittima da svolgere – e rende peraltro implausibile la tecnica di abuso più nota di questo LOLBin (l'esecuzione di una classe decorata con l'attributo `[RunInstaller(true)]`), che richiederebbe invece il percorso di un assembly come argomento. Il terzo elemento, decisivo, è il comportamento successivo, descritto nella sezione seguente. ANY.RUN rileva inoltre la firma di .NET Reactor proprio su questo processo, a conferma che al suo interno è stato caricato codice offuscato estraneo.

4.5 Comando e controllo

Il processo `InstallUtil.exe` (PID 5152) stabilisce una connessione di rete verso un endpoint esterno:

Tabella 4: Endpoint di comando e controllo identificato.

Attributo	Valore
Dominio	<code>egehgdeshbjhtre.duckdns.org</code>
Indirizzo IP	<code>91.92.253.47</code>
Porta	<code>7702 (TCP)</code>
Paese	Bulgaria (BG)
Reputazione ANY.RUN	<i>unknown</i>
Processo	<code>InstallUtil.exe</code> (PID 5152)

Ogni elemento di questa riga è un indicatore in sé.

Il dominio appartiene al servizio **DuckDNS**, una piattaforma Dynamic DNS gratuita. I servizi DDNS permettono di associare un nome host a un indirizzo IP variabile e di modificare tale associazione in qualsiasi momento: per un attaccante significa poter spostare l'infrastruttura senza dover aggiornare i campioni già distribuiti, e poter sopravvivere al sinkholing di un singolo indirizzo IP. Sono strumenti pienamente legittimi, ma il loro uso da parte di un processo di sistema costituisce un'anomalia significativa in un contesto aziendale.

Il nome scelto, `egehgdeshbjhtre`, è una stringa priva di significato, generata casualmente. È un tratto ricorrente nell'infrastruttura malevola, dove il nome del dominio non deve essere memorabile ma soltanto univoco, e costituisce un buon candidato per euristiche basate sull'entropia dei nomi di dominio.

La porta 7702 non corrisponde ad alcun servizio standard. Il traffico legittimo in uscita da una postazione di lavoro si concentra sulle porte 80, 443 e su pochi altri servizi noti; una connessione su porta alta arbitraria, avviata da un processo del .NET Framework, è un segnale forte. ANY.RUN lo registra esplicitamente con l'indicatore *Connects to*

unusual port. Vale la pena osservare che questo intervallo di porte è caratteristico di alcune famiglie di RAT scritte in .NET, come discusso più avanti.

Infine, l'assenza di un ASN identificato e la geolocalizzazione in Bulgaria per un host contattato da una workstation aziendale completano il quadro di anomalia.

4.6 Firme di rete Suricata

Il motore di firme di ANY.RUN ha generato 19 alert di rete, distribuiti su tre categorie riassunte nella tabella seguente.

Tabella 5: Classificazione degli alert di rete generati durante il task.

Classe	Firma	Interpretazione
Potentially Bad Traffic	ET INFO DYNAMIC_DNS Query to a *.duckdns.org Domain	Risoluzione di un dominio Dynamic DNS, tipica dell'infrastruttura C2
Misc activity	ET INFO DYNAMIC_DNS Query to *.duckdns. Domain	Variante della firma precedente, a minore severità
Not Suspicious Traffic	INFO [ANY.RUN] Attempting to access raw user content on GitHub	Accesso ai contenuti raw di GitHub, canale di consegna del payload

Un aspetto merita attenzione perché rappresenta un classico errore di interpretazione in fase di triage: tutti gli alert sono attribuiti al PID 2256, che corrisponde a `svchost.exe -k NetworkService -p -s Dnscache`. Questo processo non è compromesso: si tratta del servizio *DNS Client* di Windows, che effettua le risoluzioni per conto di tutte le applicazioni del sistema. Il processo che ha realmente richiesto la risoluzione è `InstallUtil.exe`, come conferma la tabella delle connessioni TCP. Attribuire l'evento a `svchost.exe` porterebbe a conclusioni errate; la correlazione tra la tabella DNS e quella delle connessioni è quindi indispensabile.

Si nota inoltre che la severità assegnata dalle firme è bassa: nessun alert è classificato come *Malicious*. Le regole Emerging Threats relative ai domini DDNS sono infatti di tipo informativo, poiché DuckDNS ha usi ampiamente legittimi. È l'analista a dover attribuire il peso corretto all'indicatore sulla base del contesto, ovvero, il fatto che la query provenga da un processo .NET avviato da un file appena scaricato.

4.7 Connessione anomala verso l'infrastruttura di GitHub

Un dato che emerge solo dalla tabella delle connessioni, e che sfugge a una lettura superficiale dell'albero dei processi, è il seguente: sia `Jvczfhe.exe` (PID 7492) sia `Muadnrd.exe` (PID 7824) stabiliscono in proprio una connessione TLS verso `avatars.githubusercontent.com` all'indirizzo `185.199.110.133:443`. Non si tratta di traffico del browser: sono i due eseguibili malevoli a contattare direttamente l'infrastruttura di GitHub.

Il dato è di difficile interpretazione, perché l'assenza di un proxy MITM impedisce di osservare il contenuto scambiato: si conosce la destinazione della connessione, non il contenuto della richiesta né se un vero scambio applicativo abbia avuto luogo. Sono plausibili almeno tre spiegazioni, non reciprocamente esclusive e non distinguibili con i dati disponibili. La prima è un semplice controllo di connettività: molte famiglie di malware verificano la presenza di un accesso reale a Internet contattando un dominio popolare e pressoché sempre raggiungibile, per accertarsi di non trovarsi in un ambiente di analisi isolato. La seconda è l'uso di un servizio legittimo come canale ausiliario per il recupero di configurazione (tecnica MITRE ATT&CK T1102, *Web Service*), ad esempio tramite dati nascosti in una risorsa ospitata sulla piattaforma. La terza è un comportamento innocuo legato a una libreria o a un componente .NET incluso nel loader, indipendente

dalla logica malevola. In assenza di visibilità sul traffico cifrato, nessuna di queste ipotesi può essere confermata o esclusa sulla base del solo task ANY.RUN.

Su 35 308 eventi di registro complessivi, solo 140 sono operazioni di scrittura. Il rapporto è di per sé indicativo: il campione legge molto e scrive pochissimo, profilo tipico di un loader che raccoglie informazioni sull'ambiente prima di decidere come procedere.

Le scritture attribuibili ai due campioni si concentrano su due aree, entrambe da interpretare con cautela perché riconducibili a comportamenti standard dello stack di rete .NET più che a scelte deliberate del malware.

La prima riguarda la disattivazione delle tracce di rete:

```
HKLM\SOFTWARE\WOW6432Node\Microsoft\Tracing\Jvczfhe_RASAPI32
    EnableFileTracing      = 0
    EnableAutoFileTracing  = 0
    EnableConsoleTracing   = 0
HKLM\SOFTWARE\WOW6432Node\Microsoft\Tracing\Jvczfhe_RASMANCS
(medesimi valori)
```

Chiavi analoghe vengono create per Muadnrd. ANY.RUN etichetta questo comportamento come *Disables trace logs* e lo classifica tra le attività sospette. Una lettura rigorosa impone però una precisazione: queste chiavi vengono create automaticamente dal sottosistema RAS ogni volta che un'applicazione .NET a 32 bit utilizza le API di rete, e i valori a zero rappresentano il comportamento predefinito. L'indicatore è quindi debole se preso isolatamente; il suo valore reale è di conferma, poiché attesta che entrambi i campioni sono binari a 32 bit (WOW6432Node) che fanno uso attivo dello stack di rete.

La seconda area riguarda la configurazione delle zone di sicurezza di Internet Explorer:

```
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMap
    ProxyBypass      = 1
    IntranetName      = 1
    UNCAsIntranet    = 1
    AutoDetect        = 0
```

Queste scritture corrispondono agli indicatori *Reads security settings of Internet Explorer* e *Checks proxy server information*. Anche in questo caso è necessaria prudenza interpretativa: le chiavi ZoneMap vengono inizializzate automaticamente da WinINet alla prima chiamata HTTP effettuata tramite le classi di rete standard del .NET Framework (*WebClient*, *HttpWebRequest*), indipendentemente dall'intento dell'applicazione chiamante. La loro presenza conferma quindi che il campione ha effettuato almeno una richiesta di rete tramite lo stack .NET, ma non costituisce di per sé una prova di ricognizione deliberata della configurazione proxy aziendale.

Da segnalare infine l'indicatore *Checks Windows Trust Settings*, associato a entrambi i campioni. Anche questo controllo rientra nella normale validazione della catena di certificati (X.509) eseguita da qualsiasi client TLS .NET, e non è di per sé prova di intento malevolo. Resta comunque un'ipotesi plausibile, nota in letteratura, che tale verifica venga sfruttata anche per rilevare la presenza di strumenti di ispezione TLS (es. proxy di intercettazione come Fiddler o Burp Suite) installati sulla macchina di analisi; i dati disponibili non permettono però di distinguere le due possibilità.

4.8 File droppati e identificazione degli hash reali

La sezione *Dropped files* della piattaforma ANY.RUN consente di isolare i file rilevanti tra le centinaia di artefatti generati da Firefox. Gli elementi significativi sono sei.

```
C:\Users\admin\Downloads\Jvczfhe.exe (executable)
MD5 5EC4256E6A2367502A8058F4BC8F4ECC
SHA256 E6A7AAFF54EB6D06ACFC6F1DFA21A85B767DBF7FF3E9BFDF2DDBDECED86AA9B2

C:\Users\admin\Downloads\00D5yt-b.exe.part (executable)
MD5 5EC4256E6A2367502A8058F4BC8F4ECC <-- identico a Jvczfhe.exe

C:\Users\admin\Downloads\Muadnrd.exe (executable)
MD5 9773175646F2942573BB40551B142A99
SHA256 B662E7213F4985684439E655BD92EA4B9A1566E76712BB86E1238113A35B90A0

C:\Users\admin\Downloads\xtorOyHX.exe.part (executable)
MD5 9773175646F2942573BB40551B142A99 <-- identico a Muadnrd.exe

C:\Users\admin\Downloads\Jvczfhe.exe:Zone.Identifier (text)
MD5 DCE5191790621B5E424478CA69C47F55

C:\Users\admin\Downloads\Muadnrd.exe:Zone.Identifier (text)
MD5 DCE5191790621B5E424478CA69C47F55 <-- identico al precedente
```

I file con estensione `.part` e nome casuale sono i file temporanei creati da Firefox durante il download e successivamente rinominati: la corrispondenza esatta degli hash lo conferma. I due flussi `Zone.Identifier`, uno per ciascun download, condividono lo stesso hash: si tratta del già citato *Mark of the Web*, il cui contenuto è generico e non dipende dal file a cui è associato.

Sono inoltre stati generati i dump di crash dei due processi, artefatti utili in un contesto di risposta agli incidenti perché contengono la memoria del processo al momento della terminazione e potrebbero quindi custodire il payload decifrato:

```
C:\Users\admin\AppData\Local\CrashDumps\Jvczfhe.exe.7492.dmp
C:\Users\admin\AppData\Local\CrashDumps\Muadnrd.exe.7824.dmp
C:\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_Jvczfhe.exe...\
Report.wer
C:\ProgramData\Microsoft\Windows\WER\ReportArchive\AppCrash_Muadnrd.exe...\
Report.wer
```

4.9 Verifica su VirusTotal e discrepanza degli hash

L'intestazione del report ANY.RUN riporta come oggetto principale del task i seguenti indicatori:

MD5	00B5E91B42712471CDFBDB37B715670C
SHA1	D9550361E5205DB1D2DF9D02CC7E30503B8EC3A2
SHA256	0307EE805DF8B94733598D5C3D62B28678EAEADBF1CA3689FA678A3780DD3DF0

La verifica di tale hash su VirusTotal ha prodotto un risultato inatteso: nessuno dei 62 motori antivirus lo segnala come malevolo (0/62). La scheda *Details* chiarisce la causa dell'anomalia.

Tabella 6: Proprietà del file corrispondente all'hash riportato da ANY.RUN.

Attributo	Valore
Nome	xtor0yHX.exe.part
Tipo di file	Text
Magic	ASCII text, with no line terminators
Dimensione	56 byte
Rilevamenti	0 / 62
Prima sottomissione	17 dicembre 2024

Un file di 56 byte di puro testo ASCII non può in alcun modo essere l'eseguibile .NET da circa 106 KB osservato in esecuzione. Il confronto con la sezione *Dropped files* chiarisce l'origine dell'anomalia: lo stesso nome temporaneo, `xtor0yHX.exe.part`, compare anche come voce di tipo eseguibile, con hash identico a quello di `Muadnrd.exe`. Firefox genera nomi temporanei per i download in corso e li riutilizza in sequenza; il dato indica quindi che tale nome è stato assegnato prima a un download interrotto (il frammento di 56 byte) e successivamente a quello completato con successo, poi rinominato in `Muadnrd.exe`. Il contenuto del frammento non è stato ispezionato in questa sede, per cui non è possibile stabilire con certezza a cosa corrisponda; è inoltre da notare che la prima sottomissione su VirusTotal risale al 17 dicembre 2024, quattro mesi dopo l'esecuzione del task, il che rende plausibile che si tratti di un frammento generico – ad esempio un errore di rete troncato – ricorrente in più occasioni indipendenti, e non necessariamente prodotto da questa sessione specifica.

La conclusione operativa resta comunque netta, ed è probabilmente l'insegnamento più utile di questo esercizio: **l'hash riportato nell'intestazione del report ANY.RUN non deve essere utilizzato come indicatore di compromissione**. Adottarlo comporterebbe due errori gravi. Il primo è produrre un falso negativo, poiché la verifica su VirusTotal restituisce 0/62 e potrebbe indurre a chiudere l'analisi ritenendo il campione innocuo. Il secondo è alimentare i sistemi di rilevamento con un indicatore inutile, lasciando i payload reali privi di copertura. Gli hash da utilizzare sono esclusivamente quelli estratti dalla sezione *Dropped files* e riportati nella sottosezione precedente.

Questo caso illustra bene un principio metodologico generale: gli strumenti automatici forniscono dati, non conclusioni. Ogni indicatore va validato incrociando almeno due fonti indipendenti prima di essere diffuso.

I campi sono stati deliberatamente contraffatti per far apparire i file come componenti del browser Microsoft Edge. Un analista che si limitasse a osservare la colonna “Descrizione” in Gestione Attività, o un operatore SOC che scorresse rapidamente un elenco di processi, potrebbe considerarli legittimi. La contraffazione è però smentita da due elementi direttamente riscontrabili nei dati del task: il percorso di esecuzione, che è la cartella Downloads dell’utente anziché **Program Files** (dove risiederebbe una reale installazione di Edge), e il nome del file, casuale e privo di relazione con Edge. A questi si aggiunge un elemento con un margine di certezza minore, perché non esplicitamente verificabile dai dati del task: è altamente plausibile che i due eseguibili non possiedano una firma digitale Authenticode valida rilasciata da Microsoft, poiché i campi Company/Description/Version sono semplici metadati del resource PE, liberamente editabili in fase di compilazione e privi di qualunque legame crittografico con l’identità dichiarata. La lezione resta comunque valida: i metadati PE sono campi arbitrari, mentre il percorso di esecuzione è un criterio di attendibilità incomparabilmente più solido.

Tabella 7: Cronologia ricostruita degli eventi principali della sessione.

Tempo	Evento
0 s	Avvio del task; explorer.exe lancia Firefox sull’URL GitHub
3,7 s	Prime richieste HTTP del browser (verifica del captive portal, OCSP)
14,5 s	Primi alert informativi relativi all’accesso ai contenuti raw di GitHub
—	Download di Jvczfhe.exe e Muadnrd.exe nella cartella Downloads
—	Esecuzione manuale dei due binari (processi 7492 e 7824)
+21 s	Ritardo artificiale tramite <code>cmd /c timeout 21</code> per ciascun campione
—	Avvio di InstallUtil.exe (PID 5152) e iniezione del payload
55,6 s	Prime query DNS verso *.duckdns.org ; alert <i>Potentially Bad Traffic</i>
—	Connessione TCP verso 91.92.253.47:7702
—	Auto-generazione della seconda istanza di Muadnrd.exe (PID 7248)
—	Crash dei due campioni; intervento di WerFault.exe
315 s	Termine della registrazione

4.10 Terminazione anomala e interpretazione dell’exit code

Entrambi i campioni terminano con un crash gestito dal sistema:

```
C:\WINDOWS\SysWOW64\WerFault.exe -u -p 7492 -s 2676 (Jvczfhe.exe)
C:\WINDOWS\SysWOW64\WerFault.exe -u -p 7824 -s 2888 (Muadnrd.exe)
```

Entrambi i processi riportano il medesimo codice di uscita: **3762504530**, che convertito in esadecimale corrisponde a **0xE0434352**. Si tratta di un valore ampiamente documentato: è il codice di eccezione strutturata (SEH) generato dal Common Language Runtime .NET quando un’eccezione gestita a livello di codice risale l’intero stack senza essere intercettata da alcun blocco `catch` dell’applicazione, comunemente indicato nella documentazione Microsoft come “*CLR Exception E0434352*”. Non identifica un tipo specifico di eccezione, ma è un wrapper generico: l’informazione sulla causa reale (il tipo di **System.Exception** sollevato e il relativo stack trace) risiederebbe nel crash dump, non nel solo exit code. Il dato conferma comunque in modo indipendente che i due campioni sono applicazioni

.NET e che la loro esecuzione è terminata per un errore applicativo non gestito, non per un intervento esterno.

Le cause plausibili sono tre, non mutuamente esclusive. La prima è un fallimento della routine di iniezione in `InstallUtil.exe`, dovuto a un disallineamento tra l'architettura del loader (32 bit, come attestato dal percorso `SysWOW64`) e quella attesa dal payload. La seconda è un meccanismo anti-analisi deliberato: alcuni campioni sollevano intenzionalmente un'eccezione quando rilevano di essere in esecuzione in un ambiente virtualizzato, interrompendosi prima di rivelare le proprie funzionalità. La terza, più prosaica, è un semplice difetto di programmazione del loader.

Il crash riguarda il processo padre (`Jvczfhe.exe` / `Muadnrd.exe`, PID 7492 e 7824), mentre l'attività di rete verso il C2 è generata dal processo figlio `InstallUtil.exe`, tecnicamente autonomo una volta avviato. I dati del task non includono un timestamp puntuale per l'evento di crash, per cui la sequenza cronologica esatta tra il contatto con il C2 e la terminazione del processo padre non è verificabile con certezza assoluta; resta però un'ipotesi plausibile, coerente con la tecnica di iniezione descritta, che l'iniezione e il contatto con il C2 abbiano preceduto la terminazione del loader originario. In ogni caso, l'analisi non ha potuto osservare le fasi eventualmente successive alla terminazione — download di moduli aggiuntivi, installazione di meccanismi di persistenza, esfiltrazione di dati — per cui la sequenza osservata resta incompleta.

4.11 Indicatori di compromissione

Si riportano di seguito gli indicatori estratti dall'analisi, distinti per tipologia. Si ribadisce che l'hash dichiarato nell'intestazione del report ANY.RUN è escluso in quanto non corrispondente al payload.

Tabella 8: Indicatori di compromissione estratti dall'analisi.

Tipo	Valore
MD5	5EC4256E6A2367502A8058F4BC8F4ECC (<code>Jvczfhe.exe</code>)
MD5	9773175646F2942573BB40551B142A99 (<code>Muadnrd.exe</code>)
Dominio C2	egehgdeshbjhtre.duckdns.org
Indirizzo IP	91.92.253.47
Porta	7702/TCP
URL di consegna	github.com/MELITERRER/frew/blob/main/Jvczfhe.exe
URL di consegna	github.com/MELITERRER/kioluu/blob/main/Muadnrd.exe
Account	MELITERRER (GitHub)
Percorso	%USERPROFILE%\Downloads\[nomecasuale].exe
Riga di comando	cmd /c timeout 21 & exit
Riga di comando	InstallUtil.exe senza argomenti

I due SHA-256 completi, per l'inserimento in sistemi di rilevamento, sono:

Jvczfh.exe

E6A7AAFF54EB6D06ACFC6F1DFA21A85B767DBF7FF3E9BFDF2DDBDECED86AA9B2

Muadnrd.exe

B662E7213F4985684439E655BD92EA4B9A1566E76712BB86E1238113A35B90A0

Tra questi indicatori, quelli comportamentali hanno valore superiore a quelli atomici. Gli hash sono facilmente eludibili con una ricompilazione, e il dominio DDNS può essere sostituito in pochi minuti; la sequenza “eseguibile in Downloads che avvia **InstallUtil.exe** senza argomenti” è invece un pattern strutturale, molto più costoso da modificare per l'attaccante e quindi più efficace come regola di rilevamento.

4.12 Ipotesi sulla famiglia di appartenenza

I dati raccolti non consentono un'attribuzione certa, poiché l'offuscamento ha impedito sia il riconoscimento per firma sia l'estrazione della configurazione – la sezione *Malware configuration* del report risulta infatti vuota. È tuttavia possibile formulare un'ipotesi ragionata, con margini di incertezza espliciti, sulla base del profilo comportamentale.

Lo sviluppo in .NET, l'uso di un dominio Dynamic DNS gratuito e la protezione tramite un offuscatore commerciale sono un profilo ricorrente nella famiglia di RAT open source derivati da **Quasar**, tra cui **AsyncRAT**, **DcRat** e **VenomRAT**: strumenti che condividono un'architettura comune, ampiamente diffusi in campagne opportunistiche di basso livello poiché il codice sorgente è pubblicamente disponibile e la configurazione del server C2 richiede competenze minime. Il dato relativo alla porta va però trattato con cautela: le versioni pubbliche di AsyncRAT preimpostano nel builder tre porte di default specifiche (6606, 7707, 8808), mentre Quasar utilizza di default la porta 4782 – valori puntuali, non un intervallo continuo. La porta osservata nel task, 7702, è vicina a 7707 ma non coincide con alcuno di questi default noti; si tratta quindi al più di un indizio debole, non di un elemento di conferma. Analogamente, l'uso di **InstallUtil.exe** come vettore di esecuzione è una tecnica LOLBin generica, adottata da numerosi loader e crypter indipendentemente dal payload finale: non è quindi un tratto distintivo di questa specifica famiglia di RAT.

Il profilo complessivo della campagna – hosting gratuito su GitHub, infrastruttura DDNS gratuita, nomi di file generati casualmente, nessuna specializzazione del bersaglio – suggerisce comunque un attore non sofisticato e una distribuzione di massa, più che un'operazione mirata. L'attribuzione a una famiglia specifica resta un'ipotesi debole, che richiederebbe, per essere confermata, l'analisi statica del payload deoffuscato o l'estrazione della configurazione dal dump di memoria.

5 Conclusioni

L'esercizio ha permesso di ricostruire in gran parte la catena di attacco di un loader .NET distribuito tramite GitHub. Il campione utilizza un insieme di tecniche coerente e ben articolato: consegna attraverso una piattaforma ad alta reputazione per superare i filtri perimetrali, contraffazione dei metadati per confondere l'ispezione superficiale, ritardo artificiale per esaurire la finestra di osservazione delle sandbox automatiche, esecuzione indiretta tramite un binario di sistema firmato per eludere i controlli basati su whitelist, offuscamento commerciale per impedire l'analisi statica, e infine infrastruttura Dynamic DNS su porta non standard per rendere resiliente il canale di comando e controllo. Nessuna di queste tecniche è individualmente innovativa, ma la loro combinazione risulta efficace contro le difese basate su un singolo livello di rilevamento.